UNIVERSIDADE ESTADUAL DO SUDOESTE DA BAHIA – UESB

DEPARTAMENTO DE CIÊNCIAS EXATAS – DCET | CURSO DE CIÊNCIA DA COMPUTAÇÃO
DISCIPLINA: COMPILADORES (2026.2) – PROF. JOSÉ CARLOS MARTINS OLIVEIRA

DISCENTE: GUILHERME DIAS SOUSA

FORMALIZAÇÃO DA GRAMÁTICA LIVRE DE CONTEXTO G = (V, T, P, S)

1. Definição Formal de Gramática Livre de Contexto

Uma **Gramática Livre de Contexto (GLC)** é definida formalmente por uma 4-tupla ***G = (V, T, P, S)***, onde:
• **V**: Conjunto finito de **símbolos não-terminais** (variáveis sintáticas).
• **T**: Conjunto finito de **símbolos terminais** (tokens e itens léxicos), sendo $V \cap T = \emptyset$.
• **P**: Conjunto finito de **regras de produção** na forma canônica $A \rightarrow \alpha$, com $A \in V$ e $\alpha \in (V \cup T)^*$. O símbolo ε denota a sentença vazia.
• **S**: **Símbolo inicial** da gramática, onde $S \in V$ (nesta linguagem, $S = PROG$).

A especificação presente na proposta do trabalho utiliza **EBNF (Forma de Backus-Naur Estendida)**, composta por metasímbolos de opcionalidade ([A]), repetição zero ou mais vezes ({B}*), repetição uma ou mais vezes ({C}+) e intercalação ({C || D}+). Para a formalização clássica de uma GLC pura, cada um desses operadores foi integralmente desdobrado em não-terminais recursivos e derivações para a sentença vazia (ε-produções).

2. Conjunto de Símbolos Terminais (T)

O conjunto **T** engloba todas as palavras reservadas, operadores, delimitadores pontuais e as classes léxicas produzidas pelo analisador léxico:

Categoria	Símbolos Terminais
Palavras Reservadas	`"program", "var", "procedure", "begin", "end", "if", "then", "else", "while", "do", "div", "mod"`
Delimitadores / Pontuação	`";", ".", ":", ",", "(", ")"`
Operadores Atribuição / Relacionais	`":=", "=", "<", ">", "<>", "<=", ">="`
Operadores Aritméticos	`"+", "-", "*"`
Classes Léxicas (Tokens)	`ident, tipo, proc, numero, variavel`

3. Conjunto de Símbolos Não-Terminais (V)

O conjunto **V** totaliza **28 não-terminais** (8 originais e 20 introduzidos para desdobrar a EBNF):
V = { PROG, BLOCO, PALIST, COMANDO, EXPR, SIEXPR, TERMO, FATOR, SECAO_VAR, LISTA_DECL_VAR, DECL_VAR, LISTA_IDENT, RESTO_IDENT, SECAO_PROC, DECL_PROC, LISTA_COMANDOS, RESTO_COMANDOS, LISTA_PARAM, RESTO_PARAM, SECAO_PARAM, OPC_VAR, LISTA_ID_PARAM, RESTO_ID_PARAM, CHAMADA_PROC, LISTA_EXPR, RESTO_EXPR, ELSE_OPC, OPC_REL, OP_REL, OPC_SINAL, RESTO_TERMO, OP_AD, RESTO_FATOR, OP_MULT }

4. Símbolo Inicial (S)

S = PROG

5. Conjunto de Regras de Produção (P)

A seguir constam as **65 produções** que definem integralmente a sintaxe da linguagem Mini_Pascal sem o uso de operadores de repetição:

PROGRAMA PRINCIPAL		
(1) **PROG**	→	`"program" ident ";" BLOCO "."`
BLOCO E DECLARAÇÕES		
(2) **BLOCO**	→	`SECAO_VAR SECAO_PROC "begin" LISTA_COMANDOS "end"`
(3) **SECAO_VAR**	→	`"var" LISTA_DECL_VAR SECAO_VAR`
(4) **SECAO_VAR**	→	ε
(5) **LISTA_DECL_VAR**	→	`DECL_VAR LISTA_DECL_VAR`
(6) **LISTA_DECL_VAR**	→	`DECL_VAR`

(7) **DECL_VAR**	→	LISTA_IDENT ":" tipo ";"
(8) **LISTA_IDENT**	→	ident RESTO_IDENT
(9) **RESTO_IDENT**	→	"," ident RESTO_IDENT
(10) **RESTO_IDENT**	→	ε
PROCEDIMENTOS		
(11) **SECAO_PROC**	→	DECL_PROC SECAO_PROC
(12) **SECAO_PROC**	→	ε
(13) **DECL_PROC**	→	"procedure" ident PALIST ";" BLOCO ";"
PARÂMETROS FORMAIS		
(14) **PALIST**	→	"(" LISTA_PARAM ")"
(15) **PALIST**	→	ε
(16) **LISTA_PARAM**	→	SECAO_PARAM RESTO_PARAM
(17) **RESTO_PARAM**	→	";" SECAO_PARAM RESTO_PARAM
(18) **RESTO_PARAM**	→	ε
(19) **SECAO_PARAM**	→	OPC_VAR LISTA_ID_PARAM ":" tipo
(20) **OPC_VAR**	→	"var"
(21) **OPC_VAR**	→	ε
(22) **LISTA_ID_PARAM**	→	ident RESTO_ID_PARAM
(23) **RESTO_ID_PARAM**	→	"," ident RESTO_ID_PARAM
(24) **RESTO_ID_PARAM**	→	ε
SEQUÊNCIA DE COMANDOS		
(25) **LISTA_COMANDOS**	→	COMANDO ";" RESTO_COMANDOS
(26) **RESTO_COMANDOS**	→	COMANDO ";" RESTO_COMANDOS
(27) **RESTO_COMANDOS**	→	ε
COMANDOS		
(28) **COMANDO**	→	variavel ":=" EXPR
(29) **COMANDO**	→	proc CHAMADA_PROC
(30) **COMANDO**	→	"if" EXPR "then" COMANDO ELSE_OPC
(31) **COMANDO**	→	"while" EXPR "do" COMANDO
(32) **COMANDO**	→	"begin" LISTA_COMANDOS "end"
CHAMADA DE PROCEDIMENTO		
(33) **CHAMADA_PROC**	→	"(" LISTA_EXPR ")"
(34) **CHAMADA_PROC**	→	ε
(35) **LISTA_EXPR**	→	EXPR RESTO_EXPR
(36) **RESTO_EXPR**	→	"," EXPR RESTO_EXPR
(37) **RESTO_EXPR**	→	ε
CONDICIONAL		
(38) **ELSE_OPC**	→	"else" COMANDO
(39) **ELSE_OPC**	→	ε
EXPRESSÕES E RELACIONAIS		
(40) **EXPR**	→	SIEXPR OPC_REL
(41) **OPC_REL**	→	OP_REL SIEXPR
(42) **OPC_REL**	→	ε
(43) **OP_REL**	→	"="
(44) **OP_REL**	→	"<"
(45) **OP_REL**	→	">"
(46) **OP_REL**	→	"<>"
(47) **OP_REL**	→	"<="
(48) **OP_REL**	→	">="
EXPRESSÕES SIMPLES		
(49) **SIEXPR**	→	OPC_SINAL TERMO RESTO_TERMO
(50) **OPC_SINAL**	→	"+"
(51) **OPC_SINAL**	→	"-"
(52) **OPC_SINAL**	→	ε
(53) **RESTO_TERMO**	→	OP_AD TERMO RESTO_TERMO

(54) **RESTO_TERMO**	→	ε
(55) **OP_AD**	→	"+"
(56) **OP_AD**	→	"-"
TERMOS E FATORES		
(57) **TERMO**	→	FATOR RESTO_FATOR
(58) **RESTO_FATOR**	→	OP_MULT FATOR RESTO_FATOR
(59) **RESTO_FATOR**	→	ε
(60) **OP_MULT**	→	"*"
(61) **OP_MULT**	→	"div"
(62) **OP_MULT**	→	"mod"
(63) **FATOR**	→	numero
(64) **FATOR**	→	variavel
(65) **FATOR**	→	"(" EXPR ")"

6. Tabela de Correspondência: EBNF Original vs. GLC

A tabela a seguir demonstra a correspondência sistemática entre cada regra em notação estendida e as regras de produção correspondentes na GLC:

Regra EBNF Original	Mapeamento Formal em GLC
`PROG -> "program" ident ";" BLOCO "."`	Produção direta **(1)**.
`BLOCO -> { "var" { ident {"," ident}* ":" tipo ";"}+ }* {"procedure" ident PALIST ";" BLOCO ";"}* "begin" {COMANDO ";"}+ "end"`	Desdobrado em produções **(2) a (13)** e **(25) a (27)**. Seção de variáveis com repetição zero ou mais (SECAO_VAR), fecho positivo de itens (LISTA_DECL_VAR) e lista de identificadores (RESTO_IDENT). Seção de procedimentos com SECAO_PROC.
`PALIST -> ["(" { { ["var"] {ident || ","}+ ":" tipo } || ";" }+ ")"]`	Desdobrado em produções **(14) a (24)**. Opcionalidade via ε em PALIST, intercalação por ponto-e-vírgula em RESTO_PARAM, passagem opcional por referência em OPC_VAR, e intercalação por vírgula em RESTO_ID_PARAM.
`COMANDO -> variavel ":=" EXPR | proc ["(" {EXPR || ","}+ ")"] | "if" ... | "while" ... | "begin" ...`	Desdobrado em produções **(28) a (39)**. Alternativas diretas para cada comando. Argumentos de chamada em CHAMADA_PROC e cláusula else opcional em ELSE_OPC.
`EXPR -> SIEXPR [("=" | "<" | ">" | "<>" | "<=" | ">=") SIEXPR]`	Desdobrado em produções **(40) a (48)**. Operador relacional opcional via OPC_REL que deriva em OP_REL SIEXPR ou ε.
`SIEXPR -> ["+" | "-"] {TERMO || "+" | "-"}+`	Desdobrado em produções **(49) a (56)**. Sinal unário opcional em OPC_SINAL e intercalação por aditivos em RESTO_TERMO.
`TERMO -> {FATOR || "*" | "div" | "mod"}+`	Desdobrado em produções **(57) a (62)**. Intercalação com operadores multiplicativos através de RESTO_FATOR.
`FATOR -> numero | variavel | "(" EXPR ")"`	Mapeamento direto nas produções **(63) a (65)**.